TSOGO-BIS

System architecture, data flows, and operations

System guide | Prepared 24 October 2026 | Source: local repository snapshot

Purpose

This guide explains how the storefront, customer and admin dashboards, database, payment providers, delivery updates, and email functions fit together. It is a codebase-level map for developers and operators.

Evidence boundary

The connected Supabase schema endpoint was unavailable during preparation. The database descriptions below were derived from checked-in schema and migration files. They are not a confirmation of the live production schema or deployment status. Files in supabase/migrations are the intended local history; unapplied files may differ from production.

1. System at a glance

TSOGO-BIS is a React single-page commerce application. Vite builds static route documents and assets; Vercel serves those documents and applies route rewrites. Supabase supplies Auth, Postgres, REST/RLS, Realtime, and Storage. Edge Functions coordinate payment gateways, account operations, email, and administrative actions.

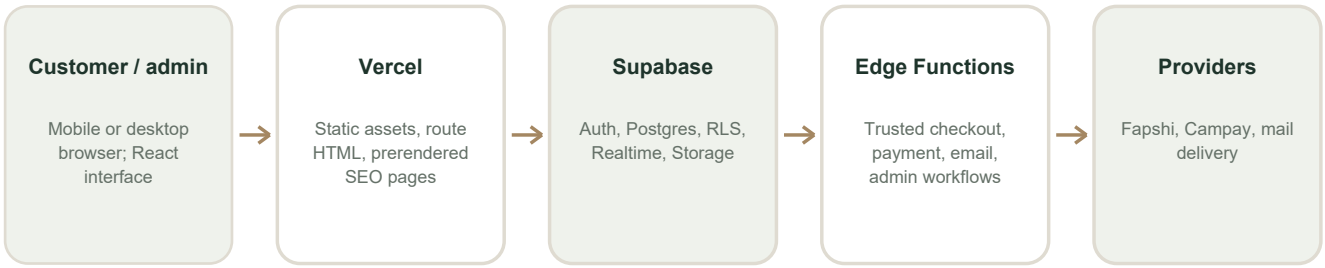


Figure 1. Main runtime boundaries. The browser never holds service-role credentials.

2. Data flow diagrams

Checkout and payment

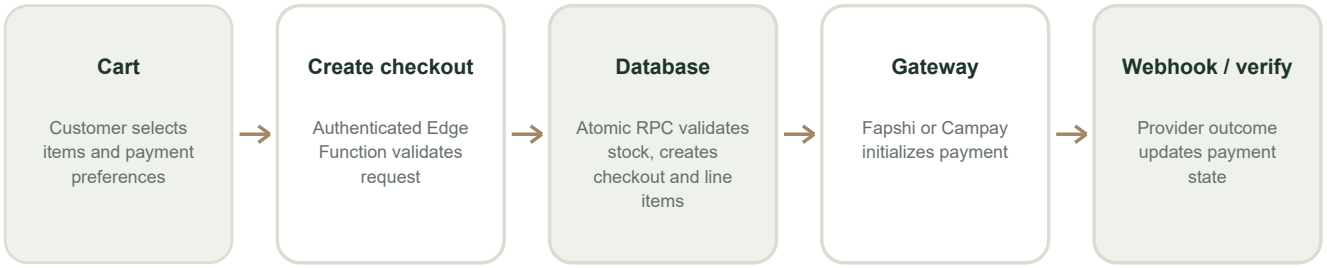


Figure 2. Checkout sequence. The local source contains both newer checkouts/checkout_items and baseline orders/order_items models; use the checkout path for the Fapshi workflow and verify the live schema before migrating or reconciling records.

Delivery status and dashboard notices

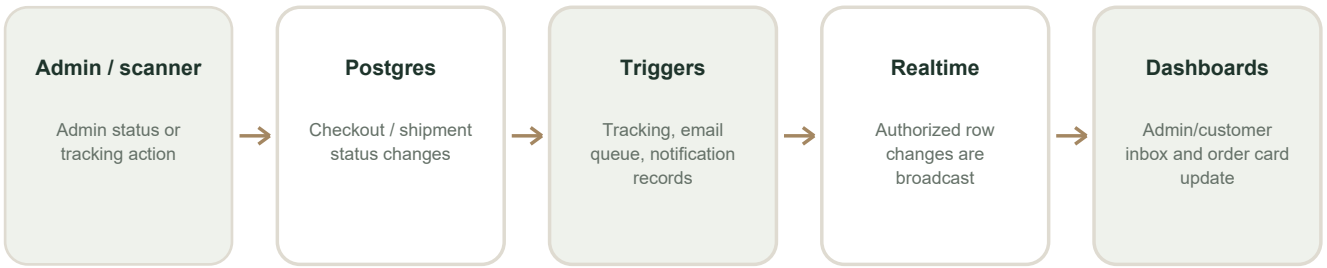


Figure 3. Status propagation. Realtime is event transport; durable state remains in Postgres.

Referral and customer credit

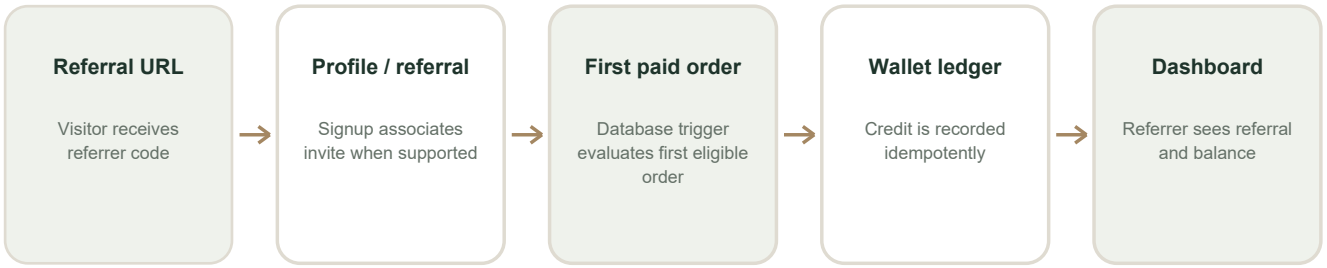


Figure 4. Referral lifecycle at a conceptual level; verify exact eligibility predicates in the live migration and current database before finance reconciliation.

3. Core data model

The baseline schema has catalog, identity, cart, order, payment, shipment, reviews, settings, and currency tables. Later migrations add the checkout transaction model, referral accounting, customer policies, coupons, email jobs, admin tools, and notifications.

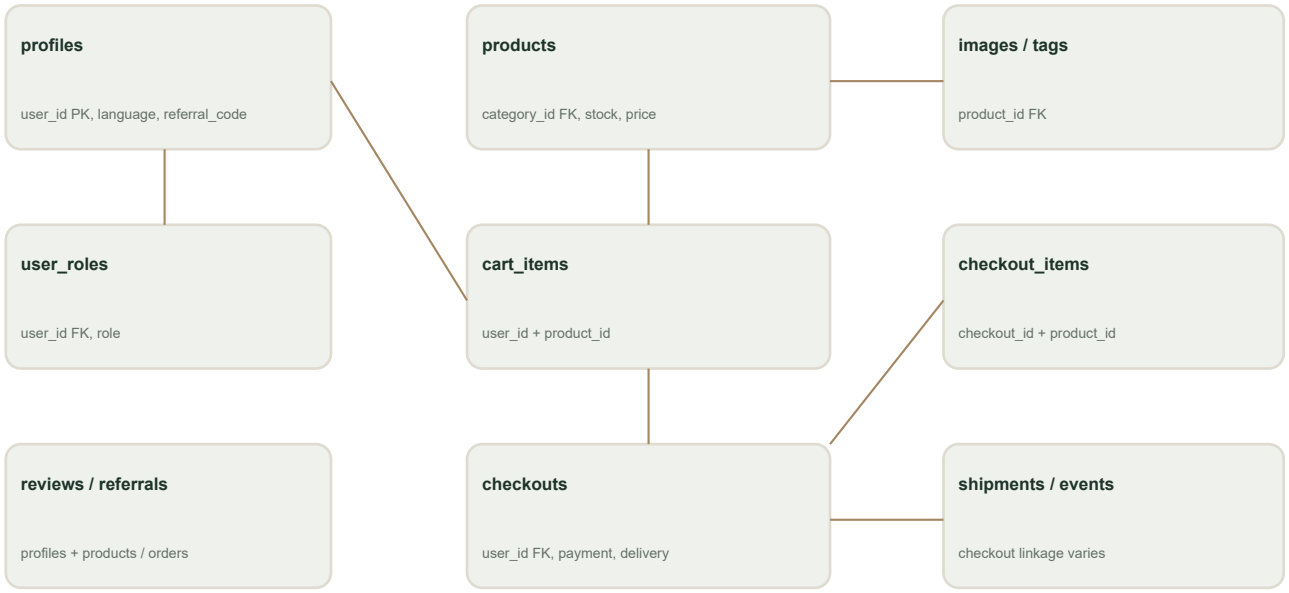


Figure 5. Simplified entity map. Arrows represent principal relationships; see the Supabase guide for table and column inventory. The initial schema-v2 order model and later checkouts model coexist in source.

4. Application surfaces

Surface	Purpose / key responsibilities
Home and shop	Catalog discovery, filters, visual image search, product links, promotions.
Product details	Options, price and delivery calculations, gallery, cart actions, public reviews.
Checkout	Customer/address details, payment mode, coupon, fees, payment initiation.
Orders and tracking	Order lookup, invoice, tracking information and status.
Customer dashboard	Account, orders, invoice downloads, cart, reviews, referrals, preferences and cookie controls.
Admin dashboard	Catalog, orders, tracking, users, reviews, referrals, coupons, ads, settings and analytics.
Auth / reset	Signup, login, admin MFA gate, reset-password flow.
Policies	Terms, privacy, delivery policies, bilingual content.
System routes	Vite prerender creates SEO-friendly pages and product metadata at build time; Vercel rewrites route paths to emitted HTML.

5. Security and trust boundaries

• Public storefront reads published catalog data through restricted Supabase policies; private order data is scoped to the authenticated customer or an authorized admin.

• Administrative access is checked server-side and the source includes an admin MFA/AAL2 gate. Client-side visibility is not the authorization boundary.

• Payment credentials and service-role keys belong in Supabase secrets. Public client environment variables may contain only the anon/publishable key and public URL.

• Webhook endpoints are unauthenticated at the Supabase JWT layer when the provider requires it; they must validate provider signatures or shared-secret credentials inside the function.

• Cookie preferences are saved in the browser. Essential storage is always active; optional analytics and preference storage follow the recorded consent.

6. Resilience and lower-bandwidth behavior

• Route-level code splitting delays checkout, admin and dashboard bundles until navigation instead of downloading every route at first paint.

• Product and catalog imagery should load lazily outside the first viewport and use async decoding. Large originals remain a main cost on 2G; use appropriately sized/compressed source images and a CDN transformation service if enabled.

• External Google font stylesheets were removed from the critical path; platform font fallbacks avoid blocking on a third-party font host.

• The static SEO fallback now includes critical inline styling while the app bundle hydrates. It remains readable while scripts or styles load slowly.

• The app is not guaranteed fully offline. Good 2G behavior still depends on reachable services, image payload size, radio latency and current device memory.

7. Build, deploy, and operate

• Build with npm run build. scripts/prerender.mjs emits static route HTML, product pages/metadata, sitemap.xml, robots.txt, llms.txt and 404.html.

• Run Supabase migrations from the repository in order after reviewing production schema state. Do not assume a local migration has been applied remotely.

• Deploy Edge Function code separately with the Supabase CLI or project deployment pipeline. Confirm the resulting version in the Supabase dashboard.

• After schema changes, validate RLS, indexes, trigger execution, Edge Function secrets, payment callbacks, Realtime subscriptions, and email provider configuration.

8. Known source-level caveats

The checked-in reference schema-v2.sql describes the original orders/order_items model, while later migrations add checkouts/checkout_items for the payment flow. The checkout tables are not a simple alias for the original order tables. Before reporting metrics or rebuilding data pipelines, establish which model is authoritative in the connected production project. The Supabase MCP read failed during this documentation run, so no live table inventory, function deployment list, extension inventory, advisor result, or migration ledger could be confirmed.

Local docs inventory

The source set inspected includes 40 migration files, supabase/schema-v2.sql, supabase/config.toml, supabase/functions, React route components, Vercel rewrites, and the Vite prerender script.